数据库设计说明

> 本文档汇总系统当前的数据存储设计，来源为代码与运行库实际定义：关系型数据见 `backend/app/core/sql_database.py`（建表 `_SCHEMA_SQL` + 幂等迁移 `_migrate()`），图数据见 `backend/app/core/database.py`。
>
> **最后校验：2026-10-08**（对照 `backend/data/app.db` 与运行中的 Neo4j 逐表 / 逐属性核对；含课程中心、题库、**L2 一题多知识点**、**L2 阶段 E 向量 BLOB**、**管理员端与审计日志**、**文档级知识融合**）
>
> 系统采用**双存储**：SQLite（关系型，存用户/课程/成员/文档/学习记录/题库/向量/平台治理与审计/**知识融合映射**）+ Neo4j（图，存知识点与关系）。

一、存储架构

存储	类型	存什么	落地文件/模块
SQLite	关系型	用户与资料、课程与成员、文档、学习记录、收藏、知识点向量、题库、**平台治理与审计日志**、**知识融合候选与映射**	`backend/app/core/sql_database.py`
Neo4j	图数据库	`KnowledgePoint` 节点 + 四类关系	`backend/app/core/database.py`

- `course_id` 三端统一为**整数**：SQLite 自增整数 → Neo4j 存整数属性 → API 传整数。
- `t_learning_record.kp_id` / `t_student_favorite.kp_id` / `t_question.kp_id` / `t_question_kp.kp_id` 均为**逻辑外键**，指向 Neo4j 的 `KnowledgePoint`，不建物理外键（跨库无法建）。
- **L2 一题多知识点（2026-09-22）**：`t_question_kp` 是题目-知识点关联的**权威数据**（一题可挂 N 个 kp），`t_question.kp_id` 退化为它的**主知识点冗余投影**——两者由 `SQLDatabase.set_question_kps()` 同事务维护，因此读侧（前端 / 统计 / 兼容期调用方）看到的 `kp_id` 语义不变。详见 2.10 / 2.15。
- 第一阶段用 SQLite（零部署、Python 标准库）；第二阶段迁移 MySQL 仅替换 `sql_database.py` 底层连接实现，表名、字段名、方法签名保持不变，业务代码零改动。
- 新增列一律走 `_migrate()` 的 `ALTER TABLE`（不写进 `CREATE TABLE`），以保证「新建库」与「迁移库」表结构不分叉——详见第七章。

二、SQLite 关系型表（22 张）

按领域分组：用户与资料（2.1–2.2）、课程与成员（2.3–2.5）、文档（2.6）、学习数据（2.7–2.9）、题库（2.10–2.12、**2.15**）、向量与缓存（2.13–2.14）、**知识融合（2.16–2.20）**、**平台治理与审计（2.21）**、级联清理（2.22）。

> 表数量以 `_SCHEMA_SQL` 实数（**22 张**，2026-10-08 核对）为准。其中 `t_question_import_batch`（Scope D 导入批次）尚未在此单列字段明细，字段定义见 `backend/app/core/sql_database.py` 与 题库功能说明.md。
>
> **相对 2026-09-23 版新增 6 张表**：管理员端的 `t_admin_audit_log`，以及知识融合的 5 张 `t_kp_fusion_*`。六张全部由 `_SCHEMA_SQL` 的 `CREATE TABLE IF NOT EXISTS` 建出，**不需要 `_migrate_xxx`**（迁移只用于给已有表加列或重建 CHECK）。融合模块的完整设计与评估见 文档级知识融合与实体消歧实施报告.md §5。

2.1 t_user 用户表

字段	类型	约束	说明
user_id	INTEGER	PK AUTOINCREMENT	主键
username	TEXT	NOT NULL UNIQUE	登录名
password_hash	TEXT	NOT NULL	密码哈希（PBKDF2-SHA256）
role	TEXT	NOT NULL DEFAULT 'student'，CHECK ∈ {admin, teacher, student}	角色（`admin` 为平台管理员，管理员端引入）
display_name	TEXT		显示名
email	TEXT		邮箱
is_active	INTEGER	NOT NULL DEFAULT 1，CHECK ∈ {0, 1}	是否启用
must_change_password	INTEGER	NOT NULL DEFAULT 0	下次登录强制改密；**仅新引导创建的账号与被管理员重置过密码的账号置 1**，历史行一律 0
created_at	TEXT	NOT NULL DEFAULT 本地时间	创建时间
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间

索引：`idx_role (role)`

> 认证相关列（`password_hash` / `username` / `role` / `is_active`）**不随个人中心改动**：扩展资料一律写入 `t_user_profile`。
>
> **`role` 的 CHECK 扩到含 `admin` 必须重建整张表**（`_migrate_admin_role()`）：SQLite 不支持修改 CHECK，与 `_migrate_question_types()` 同理，且因 `t_user` 被 `t_course` / `t_document` / `t_learning_record` 等多表引用，重建要用「关闭外键的独立连接」。`must_change_password` 则是普通列加法（`_migrate_admin_user_columns()`），`PRAGMA table_info` 守卫后 `ALTER TABLE ADD COLUMN`。
>
> ⚠️ **一处已知的结构分叉**：`must_change_password` 同时写进了 `_SCHEMA_SQL` 的 `CREATE TABLE`（带 `CHECK (must_change_password IN (0, 1))`）**和** `_migrate_admin_user_columns()` 的 `ALTER TABLE`（不带 CHECK）——这与第七章「新增列只写 `_migrate()`、不写 `CREATE TABLE`」的纪律相悖。后果：**新建库该列有 CHECK，旧库迁移来的没有**。取值合法性目前靠应用层兜底，不影响功能，但核对表结构时会看到两库不一致，勿误判为「迁移没跑」。

2.2 t_user_profile 用户资料表

主键即 `user_id`，与 `t_user` 一对一。**所有资料字段均可空**，未填写时为 `NULL`。

字段	类型	约束	说明
user_id	INTEGER	PK，FK → t_user(user_id)	用户（同时是主键）
avatar_url	TEXT		头像地址
real_name	TEXT		真实姓名
nickname	TEXT		昵称（展示优先级高于真实姓名）
gender	TEXT		性别，取值见 `GENDERS`
school	TEXT		学校
college	TEXT		学院
bio	TEXT		个人简介
student_no	TEXT		学号（学生）
major	TEXT		专业（学生）
grade	TEXT		年级（学生）
class_name	TEXT		班级（学生）
teacher_no	TEXT		教师工号（教师）
title	TEXT		职称（教师）
research_area	TEXT		研究方向（教师）
created_at / updated_at	TEXT	NOT NULL DEFAULT 本地时间	时间戳

索引：`idx_profile_student_no (student_no)`、`idx_profile_teacher_no (teacher_no)`

> 角色字段隔离由服务层白名单保证（`profile_service.py` 的 `SHARED_FIELDS` / `STUDENT_FIELDS` / `TEACHER_FIELDS`）：学生提交 `teacher_no` 会被静默丢弃，反之亦然。

2.3 t_course 课程表

字段	类型	约束	说明
course_id	INTEGER	PK AUTOINCREMENT	主键（三端统一的整数）
course_code	TEXT	UNIQUE	课程编号
course_name	TEXT	NOT NULL	课程名
description	TEXT		课程描述
teacher_id	INTEGER	NOT NULL，FK → t_user(user_id)	创建者（课程归属的唯一依据）

字段	类型	约束	说明
status	INTEGER	NOT NULL DEFAULT 1，CHECK ∈ {0, 1}	状态
created_at	TEXT	NOT NULL DEFAULT 本地时间	创建时间
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间
join_code	TEXT	可空；唯一性由独立唯一索引保证	8 位加课码
join_mode	TEXT	NOT NULL DEFAULT 'approval'，取值见 `JOIN_MODES`	加入方式
organization	TEXT		院系/机构
category	TEXT		课程分类
cover	TEXT		封面
is_public	INTEGER	NOT NULL DEFAULT 1	是否公开（可被「发现课程」搜索到）
governance_status	TEXT	NOT NULL DEFAULT 'normal'；取值见 `COURSE_GOVERNANCE_STATUS`	**平台治理状态**：`normal` / `hidden`（平台下架）/ `archived`（归档）
governance_note	TEXT		治理操作备注（管理员填写）
archived_at	TEXT	可空	归档时间

索引：`idx_teacher_id (teacher_id)`、`uq_course_join_code`（**UNIQUE**，仅 `join_code`）

> `join_code` 等 6 列由 `_migrate_course_center()` 补入，**刻意不写进 `CREATE TABLE`**（见第七章）。`join_code` 只加成普通可空列——SQLite 的 `ALTER TABLE ADD COLUMN` 不允许带 UNIQUE，唯一性交给独立的 `CREATE UNIQUE INDEX`（唯一索引允许多个 NULL，可与历史未发码的行共存）。
>
> `governance_status` / `governance_note` / `archived_at` 三列由 `_migrate_admin()` 补入，同样**不写进 `CREATE TABLE`**（新库也走这段迁移），且**不带 CHECK**——取值合法性由应用层 `COURSE_GOVERNANCE_STATUS` 白名单兜底。历史课程一律 `normal`。
>
> **治理状态与业务状态是两回事，别混用**：`status`（0/1）是**教师维度**的业务开关，`governance_status` 是**平台维度**的下架/归档。管理员「下架课程」只改 `governance_status='hidden'`，**不动 `status`**；「代教师关闭课程」才改 `status=0`。对外可见性统一走 `COURSE_VISIBLE_SQL = "status = 1 AND COALESCE(governance_status, 'normal') = 'normal'"` —— `COALESCE` 是为了兼容 `governance_status` 尚未回填的历史行。

2.4 t_course_member 课程成员表

用户 ↔ 课程的多对多关系表，**替代「把学生硬编码进课程表」的做法**。教师与学生在同一张表内，用 `role` 区分。

字段	类型	约束	说明
member_id	INTEGER	PK AUTOINCREMENT	主键
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程

字段	类型	约束	说明
user_id	INTEGER	NOT NULL，FK → t_user(user_id)	用户
role	TEXT	NOT NULL DEFAULT 'student'，CHECK ∈ {teacher, student}	课程内角色
status	TEXT	NOT NULL DEFAULT 'pending'，CHECK ∈ {pending, approved, rejected, removed}	成员状态
join_source	TEXT	NOT NULL DEFAULT 'code'，CHECK ∈ {create, code, invite, apply, import}	加入来源
applied_reason	TEXT		申请理由（发现课程页申请时填写）
reviewed_by	INTEGER	可空	审核人（教师）
reviewed_at	TEXT	可空	审核时间
review_comment	TEXT	可空	审核意见
joined_at	TEXT	可空	正式加入时间（approved 时才写入）
created_at / updated_at	TEXT	NOT NULL DEFAULT 本地时间	时间戳

约束：`UNIQUE (course_id, user_id)` ——同一用户在同一课程至多一条关系，重复加入不产生脏数据

索引：`idx_cm_course_status (course_id, status)`、`idx_cm_course_role (course_id, role)`、`idx_cm_user_status (user_id, status)`

> **状态语义**：只有 `status='approved'` 才视为课程成员并放行**内容访问**（文档/图谱/问答/学习路径）；`pending` / `rejected` / `removed` 仅有**元数据级**读取权，用于查看自己的申请状态，访问内容一律 4003。被移除的学生再次用加课码加入时，服务层强制回落为 `pending`，不因 `join_mode='auto'` 自动放行——避免泄露出去的加课码撤销教师的「移除」决定。
>
> 教师创建课程时自动写入 `role='teacher', status='approved', join_source='create'`。

2.5 t_course_invite 课程邀请表

字段	类型	约束	说明
invite_id	INTEGER	PK AUTOINCREMENT	主键
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程
token	TEXT	NOT NULL，UNIQUE	邀请令牌（随机生成，不可由 course_id/user_id 推导）
invited_by	INTEGER	NOT NULL，FK → t_user(user_id)	发起邀请的教师
role	TEXT	NOT NULL DEFAULT 'student'	被邀请者加入后的角色

字段	类型	约束	说明
status	TEXT	NOT NULL DEFAULT 'active'，CHECK ∈ {active, used, revoked}	状态
used_by	INTEGER	可空，FK → t_user(user_id)	使用者
used_at	TEXT	可空	使用时间
expires_at	TEXT	可空	过期时间（默认 7 天）
created_at	TEXT	NOT NULL DEFAULT 本地时间	创建时间

索引：`idx_invite_course (course_id, status)`、`token` 唯一索引

> 令牌消费用 `UPDATE ... WHERE status='active'` 判断 `rowcount==1` 实现原子占用，无「先读后写」竞态。刷新加课码通过覆盖 `t_course.join_code` 实现，旧码立即失效（无历史码表）。

2.6 t_document 文档表

字段	类型	约束	说明
doc_id	INTEGER	PK AUTOINCREMENT	主键
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	所属课程
uploader_id	INTEGER	NOT NULL，FK → t_user(user_id)	上传者
file_name	TEXT	NOT NULL	文件名
file_type	TEXT	NOT NULL，CHECK ∈ {PDF, TXT, DOCX, MD}	文件类型
file_size	INTEGER	NOT NULL	文件大小
file_sha256	TEXT		文件哈希
file_path	TEXT		存储路径
parse_status	TEXT	NOT NULL DEFAULT 'UPLOADED'，CHECK ∈ {UPLOADED, PARSING, PARSED, FAILED}	解析状态
extract_status	TEXT	NOT NULL DEFAULT 'PENDING'，CHECK ∈ {PENDING, EXTRACTING, COMPLETED, FAILED}	抽取状态
error_message	TEXT		错误信息
chunk_count	INTEGER	NOT NULL DEFAULT 0	分块数
entity_count	INTEGER	NOT NULL DEFAULT 0	抽取实体数
relation_count	INTEGER	NOT NULL DEFAULT 0	抽取关系数
vector_collection	TEXT		向量集合名
extract_started_at	TEXT	可空	抽取任务**开始**时刻（旧库历史行为 `NULL`）

字段	类型	约束	说明
extract_finished_at	TEXT	可空	抽取任务**结束**时刻（旧库历史行为 `NULL`）
created_at	TEXT	NOT NULL DEFAULT 本地时间	创建时间
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间

索引：`idx_course_id`、`idx_uploader_id`、`idx_parse_status`、`idx_extract_status`

> **为什么要单独记抽取起止时刻**：管理端「资源管理 → 知识抽取任务」原先用 `created_at` / `updated_at` 算耗时，而 `updated_at` 会被**任何**后续写操作刷新（回填统计、重建图谱、改元数据），于是「耗时」实际变成了「上传至今」——实测有文档显示 48889 分（34 天）。
>
> 两列由 `_migrate_document_extract_timing()` 补入，**不伪造历史值**：`t_document` 从未记录过抽取完成时刻，历史行的真实耗时无法还原，故两列留 `NULL`，前端显示「—」。之后由 `update_document` 在写 `extract_status` 的同一句 `UPDATE` 里打戳。
>
> 与 `must_change_password` 一样，这两列也**同时出现在 `_SCHEMA_SQL` 与 `_migrate()` 两处**（第七章纪律一的例外）；不同的是两处都是**无约束的可空 `TEXT`**，因此**不产生结构分叉**——不像 `must_change_password` 那样一处带 CHECK、一处不带。

2.7 t_learning_record 学习记录表

字段	类型	约束	说明
record_id	INTEGER	PK AUTOINCREMENT	主键
user_id	INTEGER	NOT NULL，FK → t_user(user_id)	用户
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程
document_id	INTEGER	可空	所属文档（文档级作用域）
kp_id	TEXT	NOT NULL	知识点（逻辑外键 → Neo4j KnowledgePoint）
status	TEXT	NOT NULL DEFAULT 'MASTERED'，CHECK ∈ {MASTERED, LEARNING, RECOMMENDED}	掌握状态
mastery_level	INTEGER	NOT NULL DEFAULT 100，CHECK BETWEEN 0 AND 100	掌握程度
source	TEXT	NOT NULL DEFAULT 'MANUAL'，CHECK ∈ {MANUAL, SYSTEM}	来源
last_learned_at	TEXT		最近学习时间
created_at / updated_at	TEXT	NOT NULL DEFAULT 本地时间	时间戳

约束：`UNIQUE (user_id, course_id, kp_id)`（同一用户对同一知识点仅一条记录）
索引：`idx_user_course (user_id, course_id)`、`idx_course_kp (course_id, kp_id)`

2.8 t_student_favorite 知识点收藏表

字段	类型	约束	说明
id	INTEGER	PK AUTOINCREMENT	主键
user_id	INTEGER	NOT NULL，FK → t_user(user_id)	学生
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程
document_id	INTEGER	可空	所属文档
kp_id	TEXT	NOT NULL	知识点（逻辑外键 → Neo4j）
created_at	TEXT	NOT NULL DEFAULT 本地时间	收藏时间

约束：`UNIQUE (user_id, course_id, kp_id)`（重复收藏幂等，靠 `INSERT OR IGNORE`）
索引：`idx_fav_user_course (user_id, course_id)`

2.9 t_kp_embedding 知识点向量表

字段	类型	约束	说明
course_id	INTEGER	PK（复合主键） NOT NULL	课程
kp_id	TEXT	PK（复合主键） NOT NULL	知识点
document_id	INTEGER	可空	所属文档
embedding	TEXT	NOT NULL	向量（JSON 数组文本，**兼容保留**）
embedding_blob	BLOB	可空	**L2 阶段 E**：float32 小端向量；检索优先读它（体积 5.55×↓、免 JSON 解析）。为空时读取回落解析 `embedding`
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间

主键：`(course_id, kp_id)`

> 无外键。`document_id` 无索引（当前检索按 `(course_id, kp_id)` 走主键）。
> `embedding_blob` 由 `_migrate_embeddings_blob()` **限量回填**（每次启动 ≤5000 行，幂等续跑）；
> 写入方（`upsert_kp_embedding`）**双写**两列，因此未回填完的历史行也能正常读。

2.10 t_question 题目表

字段	类型	约束	说明
question_id	INTEGER	PK AUTOINCREMENT	主键
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	归属课程
document_id	INTEGER	可空	空 = 课程通用题；非空 = 挂到具体文档
kp_id	TEXT	可空	**主知识点（冗余投影）**：真正的关联见 `t_question_kp`（2.15）。本列由 `set_question_kps()` 同事务同步，读侧语义不变；空值统一为 `NULL`
q_type	TEXT	NOT NULL，CHECK ∈ {SINGLE, MULTI, JUDGE, FILL, ESSAY}	题型（Scope B 扩两型主观题）
stem	TEXT	NOT NULL	题干
options	TEXT		选项/空位（JSON 文本）：选择 `[{"key":"A","text":"..."}]`；**填空** `[{"key":1,"label":"第1空","hint":"单位 kg","score":50,"answer":"浮点数"}]`
answer	TEXT	NOT NULL	答案（JSON 文本）：单选 `"A"` / 多选 `["A","C"]` / 判断 `"true"` / 填空 `["浮点数","0.5"]`（参考答案数组）/ 解答 `"参考答案正文"`
analysis	TEXT		解析（提交后展示给学生）
difficulty	INTEGER	NOT NULL DEFAULT 3，CHECK BETWEEN 1 AND 5	难度
source	TEXT	NOT NULL DEFAULT 'MANUAL'，CHECK ∈ {MANUAL, AI, IMPORT}	来源
created_by	INTEGER	NOT NULL，FK → t_user(user_id)	创建教师
is_active	INTEGER	NOT NULL DEFAULT 1，CHECK ∈ {0,1}	启用状态（0 = 停用，等价软删）
created_at / updated_at	TEXT	NOT NULL DEFAULT 本地时间	时间戳

索引：`idx_q_course_doc (course_id, document_id)`、`idx_q_kp (kp_id)`、`idx_q_type (course_id, q_type)`

> `options` / `answer` 用 JSON 文本而非拆表：题型差异大（拆表会产生大量空列）；且必须 `json.dumps` 统一序列化，否则单选答案 `"A"` 无法回读、判断答案 `"true"` 会被误读成布尔值。

2.11 t_answer_record 学生答题记录表

字段	类型	约束	说明
record_id	INTEGER	PK AUTOINCREMENT	主键
user_id	INTEGER	NOT NULL，FK → t_user(user_id)	学生
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程
document_id	INTEGER	可空	题目所属文档（课程通用题为空）
question_id	INTEGER	NOT NULL，FK → t_question(question_id)	题目
user_answer	TEXT		学生作答（JSON 文本，与 answer 同结构）
is_correct	INTEGER	NOT NULL DEFAULT 0，CHECK ∈ {0,1}	是否答对（主观题未批改时为占位 0，须按 grade_status 过滤）
score	REAL	NOT NULL DEFAULT 0	得分（客观题 0/100；主观题批改后 0~100 可为部分分）
grade_source	TEXT	NOT NULL DEFAULT 'AUTO'，CHECK ∈ {AUTO, LLM, TEACHER}	判分来源（主观题批改后为 TEACHER）
grade_status	TEXT	NOT NULL DEFAULT 'GRADED'，CHECK ∈ {PENDING, GRADED}	批改状态：主观题提交写 PENDING（不判分）
graded_by	INTEGER	可空	批改教师 user_id（逻辑外键，不设 FK：教师注销不应破坏记录）
graded_at	TEXT	可空	批改时间
comment	TEXT	可空	教师评语（学生可见）
answered_at	TEXT	NOT NULL DEFAULT 本地时间	作答时间

索引：`idx_ans_user_q (user_id, question_id)`、`idx_ans_user_c (user_id, course_id, document_id)`

> **追加式**（不建 UNIQUE）：同一题可多次作答；错题本取「每题最近一次答错」，同时保留历史，便于统计正确率趋势。
>
> **Scope B 批改是「就地更新」**：教师批改直接 UPDATE 同一行（score/is_correct/grade_status/grade_source/graded_by/graded_at/comment），不留批改历史版本；重批即覆盖。
> 旧库迁移用 `ALTER TABLE ADD COLUMN`（默认 `GRADED`，保证历史行与客观题行为零变化）；SQLite 的 ADD COLUMN 不能带 CHECK，旧库取值合法性由应用层 `ANSWER_GRADE_STATUS` 兜底。

2.12 t_question_favorite 学生题目收藏表

字段	类型	约束	说明
id	INTEGER	PK AUTOINCREMENT	主键

字段	类型	约束	说明
user_id	INTEGER	NOT NULL，FK → t_user(user_id)	学生
course_id	INTEGER	NOT NULL，FK → t_course(course_id)	课程
question_id	INTEGER	NOT NULL，FK → t_question(question_id)	题目
created_at	TEXT	NOT NULL DEFAULT 本地时间	收藏时间

约束：`UNIQUE (user_id, question_id)`（重复收藏幂等）
索引：`idx_qfav_user_course (user_id, course_id)`

2.13 t_question_embedding 题目向量表（Scope C：知识点自动标注）

字段	类型	约束	说明
question_id	INTEGER	PK	题目 id（一题一行）
course_id	INTEGER	NOT NULL	归属课程
document_id	INTEGER	可空	题目所属文档
text_hash	TEXT	NOT NULL	题干+选项的指纹；**变化才重算向量**
embedding	TEXT	NOT NULL	向量（JSON 文本，bge-m3 1024 维，**兼容保留**）
embedding_blob	BLOB	可空	**L2 阶段 E**：float32 小端向量（口径同 `t_kp_embedding.embedding_blob`）；供召回 R5 语义近邻读取
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间

索引：`idx_q_emb_scope (course_id, document_id)`。无外键，故在课程/文档删除路径显式清理。

2.14 t_kp_text 知识点文本缓存表（Scope C：图库不可用时的降级数据源）

字段	类型	约束	说明
course_id	INTEGER	NOT NULL，PK 之一	课程
kp_id	TEXT	NOT NULL，PK 之一	知识点业务主键（与 Neo4j 对齐）
document_id	INTEGER	可空	所属文档（普通列，避免 NULL 进主键）
name	TEXT	NOT NULL	知识点名称（字面匹配用）
category	TEXT		类别
description	TEXT		描述
updated_at	TEXT	NOT NULL DEFAULT 本地时间	缓存时间

索引：`idx_kp_text_scope (course_id, document_id)`。写入时机：每次从 Neo4j 成功取到知识点清单后**覆盖式更新**。

> 独立于 `t_student_favorite`（知识点收藏）：后者 `kp_id NOT NULL` 且语义为「知识点书签」，混用会污染两侧统计口径。

2.15 t_question_kp 题目-知识点关联表（L2：一题多知识点）

字段	类型	约束	说明
question_id	INTEGER	PK 之一，FK → t_question(question_id)	题目
kp_id	TEXT	PK 之一	逻辑外键 → Neo4j KnowledgePoint
score	REAL	可空	标注置信度（`kp_labeler` 自动标注产物）
source	TEXT	NOT NULL DEFAULT 'MANUAL'，CHECK ∈ {MANUAL, AI, IMPORT}	关联来源（与 `QUESTION_SOURCE` 同口径）
is_primary	INTEGER	NOT NULL DEFAULT 0，CHECK ∈ {0,1}	是否「主知识点」
created_at	TEXT	NOT NULL DEFAULT 本地时间	建立时间

主键：`(question_id, kp_id)`（同题同 kp 天然幂等）

索引：

- `idx_qkp_kp (kp_id)`——按知识点反查题目（出题 / 覆盖率）
- `uq_qkp_primary`：`UNIQUE (question_id) WHERE is_primary = 1`（**部分唯一索引**）——「一题最多一个主知识点」由 DB 层保证而非应用层自觉；SQLite 部分索引自 3.8.0 起支持

写入纪律：本表是题目-知识点关联的**唯一事实来源**，`t_question.kp_id` 是它的投影（见 2.10）。兼容期内**只有 `SQLDatabase.set_question_kps()` 一个写入口**，它在一个事务里完成「删除旧关联 → 重建新关联 → 同步投影」，并在应用层归一化（去空、去重、保序、`primary` 不在列表时兜底取首项）。`create_question` / `update_question` / `QuestionService.import_commit` 均经它落库；`update_question` **只在调用方显式传入 kp 字段时**才写关联——否则「教师只改难度」会把多 KP 题**塌缩**成单 KP。

上限：`question_service.MAX_KP_PER_QUESTION = 10`（防误传超大数组；同时为组卷的「任一命中即占用配额槽」留出上界）。

迁移：`_migrate_question_kp()` 只回填「关联表中无任何行」的题——若已有 `is_primary=1` 的行（教师后续编辑产生）再按 `t_question.kp_id` 插一行主 kp，会同时触碰 `uq_qkp_primary` 导致后端启动即崩。详见第七章。

> L2「一题多知识点」的完整设计、语义口径（决策 1/2/3/4）、验证结果与回滚方式见 题库与推荐系统设计.md §13。

2.16 t_kp_fusion_run 融合批次表

一次「文档级知识融合」执行的批次记录（DRY_RUN 试算 / APPLY 正式应用）。

字段	类型	约束	说明
run_id	TEXT	PK	批次 id（业务生成，非自增）
course_id	INTEGER	NOT NULL	课程
document_id	INTEGER	NOT NULL	文档（融合严格限定在单个文档内）
mode	TEXT	NOT NULL，CHECK ∈ {DRY_RUN, APPLY}	试算 / 正式应用
status	TEXT	NOT NULL DEFAULT 'RUNNING'，CHECK ∈ {RUNNING, COMPLETED, FAILED, REVOKED}	批次状态
total_candidates	INTEGER	NOT NULL DEFAULT 0	本批候选总数
n_same / n_different / n_uncertain	INTEGER	NOT NULL DEFAULT 0	判定为同义 / 异义 / 不确定的条数
n_applied	INTEGER	NOT NULL DEFAULT 0	实际写入映射的条数（DRY_RUN 恒为 0）
config_json	TEXT		本次阈值与配置的**快照**（保证批次可复现）
operator_id / operator_name	INTEGER / TEXT	可空	操作人
error_message	TEXT		失败原因
started_at	TEXT	NOT NULL DEFAULT 本地时间	开始时刻
finished_at / revoked_at	TEXT	可空	结束 / 撤销时刻
revoked_by	INTEGER	可空	撤销人

索引：`idx_fusion_run_scope (course_id, document_id, started_at)`

> 无物理外键（与 `t_kp_embedding` / `t_kp_text` 同一先例）：`_connect()` 默认开 `PRAGMA foreign_keys=ON`，设 FK 会让删文档的手工级联直接报错。

2.17 t_kp_fusion_candidate 融合候选表

同一文档内「两个知识点可能是同一个」的候选项及其判定结果。

字段	类型	约束	说明
candidate_id	INTEGER	PK AUTOINCREMENT	主键

字段	类型	约束	说明
course_id / document_id	INTEGER	NOT NULL	作用域（候选只在本文档内产生）
source_kp_id / target_kp_id	TEXT	NOT NULL	候选对（逻辑外键 → Neo4j `KnowledgePoint`）
source_name / target_name	TEXT		冗余存名，便于驳回记忆与列表展示
source_category / target_category	TEXT		冗余存类别（评分特征之一）
score	REAL	NOT NULL DEFAULT 0	综合得分
score_detail	TEXT		分项得分（JSON，可解释性）
decision	TEXT	NOT NULL DEFAULT 'UNCERTAIN'，CHECK ∈ {SAME, DIFFERENT, UNCERTAIN}	判定结论
decision_source	TEXT	NOT NULL DEFAULT 'RULE'，CHECK ∈ {RULE, LLM, MANUAL}	判定来源（规则 / 大模型 / 人工）
reason	TEXT		判定理由
status	TEXT	NOT NULL DEFAULT 'PENDING'，CHECK ∈ {PENDING, ACCEPTED, REJECTED, DEFERRED, SUPERSEDED}	人工处理状态
last_run_id	TEXT		最近一次产出该候选的批次
reviewed_by / reviewed_at / review_comment	INTEGER / TEXT / TEXT	可空	人工复核信息
created_at / updated_at	TEXT	NOT NULL DEFAULT 本地时间	时间戳

约束：`UNIQUE (course_id, document_id, source_kp_id, target_kp_id)`

> 该唯一约束是**不变量载体**：保证「驳回可记忆」——同一对候选重复跑不会产生第二条记录，人工已驳回的结果不会被下一批次覆盖掉。

2.18 t_kp_fusion_map 融合映射表

读图折叠的唯一数据源。融合**不修改 Neo4j**，而是在本表记一条「source → target」映射，读图时按映射把 source 折叠到 target。

字段	类型	约束	说明
fusion_id	INTEGER	PK AUTOINCREMENT	主键

字段	类型	约束	说明
course_id / document_id	INTEGER	NOT NULL	作用域
source_kp_id / target_kp_id	TEXT	NOT NULL	被合并方 / 合并目标（逻辑外键 → Neo4j）
source_name / target_name / source_category / target_category / source_description	TEXT		冗余快照（source 折叠后不再单独可读，需留档）
score	REAL		采纳时的得分
decision_source	TEXT		采纳来源（RULE / LLM / MANUAL）
reason	TEXT		采纳理由
run_id	TEXT		产出该映射的批次
candidate_id	INTEGER		对应的候选记录
status	TEXT	NOT NULL DEFAULT 'ACTIVE', CHECK ∈ {ACTIVE, REVOKED}	生效 / 已撤销
operator_id / operator_name	INTEGER / TEXT	可空	操作人
created_at	TEXT	NOT NULL DEFAULT 本地时间	建立时刻
revoked_at / revoked_by	TEXT / INTEGER	可空	撤销信息

约束：`UNIQUE (course_id, document_id, source_kp_id)` ——保证「**一个 source 只能指向一个 target**」（目标唯一），避免同一节点被折叠到两处
索引：`idx_fusion_map_scope (course_id, document_id, status)`

> 撤销走 `status='REVOKED'` 而非 `DELETE`：保留审计痕迹，且 `UNIQUE` 约束会继续拦住「已撤销的 source 重新指向新 target」以外的意外重复。

2.19 t_kp_fusion_conflict 融合冲突表

折叠映射之间的结构冲突（如折叠后形成环、一个 source 被多方争抢）。

字段	类型	约束	说明
conflict_id	INTEGER	PK AUTOINCREMENT	主键
course_id / document_id	INTEGER	NOT NULL	作用域
conflict_type	TEXT	NOT NULL，CHECK ∈ {CYCLE, MULTI_TARGET, ENDPOINT_MISSING, CROSS_DOC}	冲突类型：成环 / 多目标 / 端点缺失 / 跨文档

字段	类型	约束	说明
source_kp_id / target_kp_id	TEXT	可空	涉事知识对
detail	TEXT		冲突详情
run_id	TEXT		检出批次
status	TEXT	NOT NULL DEFAULT 'OPEN'，CHECK ∈ {OPEN, RESOLVED}	未处理 / 已解决
created_at	TEXT	NOT NULL DEFAULT 本地时间	检出时刻
resolved_at / resolved_by	TEXT / INTEGER	可空	处理信息

索引：`idx_fusion_conflict_scope (course_id, document_id, status)`

2.20 t_kp_fusion_scope 融合作用域表（epoch）

每 `(course_id, document_id)` 一行的**融合版本号**，用于判定「向量索引是否还新鲜」。

字段	类型	约束	说明
course_id / document_id	INTEGER	PK（复合）NOT NULL	作用域
epoch	INTEGER	NOT NULL DEFAULT 0	融合状态版本；每次 apply / revoke **+1**（首次调用即建行，从 1 开始）
indexed_epoch	INTEGER	可空	「向量索引已按此 epoch 建好」的水位，供 `ensure_index` 判新鲜度
updated_at	TEXT	NOT NULL DEFAULT 本地时间	更新时间

主键：`(course_id, document_id)`

> **为什么需要它**：融合生效会改变读图折叠后的 `kp_id` 集合，旧向量随之失效。`apply` / `revoke` 在**同一个 SQLite 事务**内完成「写入映射 → `epoch+1` → 失效向量」，再由 `fusion_scope_mark_indexed()` 在索引重建后抬升 `indexed_epoch`。`epoch != indexed_epoch` 即代表索引过期需重建——这与 §2.9 的懒重建是同一套新鲜度思路，但判据是版本号而非集合比对。
>
> 无记录时 `fusion_scope_get()` 返回 `None`，视为「从未融合过」，是合法的初始态。

2.21 t_admin_audit_log 管理员审计日志表

管理员端所有写操作的审计流水，**追加式**（只 INSERT，不 UPDATE / DELETE）。

字段	类型	约束	说明
log_id	INTEGER	PK AUTOINCREMENT	主键

字段	类型	约束	说明
operator_id	INTEGER	可空	操作人 user_id（**逻辑外键，不设 FK**：账号注销不应破坏审计流水）
operator_name / operator_role	TEXT	可空	操作人快照（避免后续改名 / 改角色导致流水失真）
action	TEXT	NOT NULL	动作名，取值见 `ADMIN_ACTIONS`（如 `course.hide`、`user.reset_password`）
target_type / target_id	TEXT	可空	操作对象类型与 id
result	TEXT	NOT NULL DEFAULT 'success', CHECK ∈ {success, failure}	成功 / 失败
detail	TEXT		明细（多为变更前后值 JSON）
ip / user_agent	TEXT	可空	来源信息
created_at	TEXT	NOT NULL DEFAULT 本地时间	发生时刻

索引：`idx_audit_created (created_at)`、`idx_audit_operator (operator_id)`、`idx_audit_action (action)`、`idx_audit_target (target_type, target_id)`

> `action` 与 `api/admin.py` 的调用点**一一对应**，取值集中在 `ADMIN_ACTIONS` 元组中维护，便于筛选与统计。
> `action` 本身**不带 CHECK**——动作集会随管理员端功能增长而扩展，写死 CHECK 又会撞上「SQLite 不能改 CHECK、只能重建表」的老问题。
> `result='failure'` 也会落库（含失败的越权尝试），这是审计日志与业务日志的关键差异。

2.22 级联清理（防孤儿数据）

SQLite 未开启外键级联，所有级联删除都在 `SQLDatabase.delete_course()` / 文档删除路径内按「子表 → 主表」顺序显式执行。

触发	清理范围	说明
删除题目	该题的**知识点关联** + 收藏 + 答题记录 → 题目本身	`t_question_kp` 有物理外键且连接已开 `PRAGMA foreign_keys=ON`，**必须先删关联行**否则抛 FK 错误；已被作答过的题目由服务层降级为软删（`is_active=0`）
删除文档	该文档精确挂载的题目 & 其**知识点关联**/收藏/答题记录；**融合 5 表**（run / candidate / map / conflict / scope）	课程通用题（`document_id IS NULL`）刻意保留。融合数据走 `delete_fusion_by_document(cid, doc_id)`，必须排在 `DELETE FROM t_document` 之前
删除课程	向量、知识点收藏、学习记录、**成员、邀请**、题库全部、文档、**融合 5 表** → 课程本身	顺序不可颠倒，否则留下指向已删课程的孤儿行

> 成员/邀请的清理在课程中心改造时补入：此前删课会残留 `t_course_member` 行。
> `t_question_kp` 的清理在 L2「一题多知识点」改造时补入，覆盖 4 个删除点：`delete_question` / `delete_course` / `delete_questions_by_course` / `delete_questions_by_document`。
> 融合数据的清理覆盖**两级**：文档级 `delete_fusion_by_document()`（`document_service.py`）、课程级 `delete_fusion_by_course()`（在 `SQLDatabase.delete_course()` 的事务内调用）。
>
> **融合数据曾漏清课程级（已修复，2026-10-08）**：这 5 张表最初只挂在**文档级**删除路径上——`SQLDatabase.delete_course()` 完全没引用它们，且全仓不存在 `delete_fusion_by_course()`。
> 因为它们都没设物理外键（见 §2.16 说明），漏清**不抛错**，只会静默堆积指向已删课程的孤儿行。
>
> **修复内容**：① 新增 `delete_fusion_by_course()`；② 文档级 / 课程级两个删除函数共用类常量 `_FUSION_TABLES` 清单（此前的重复字面量正是「加表时只挂一处」的温床）；③ 在 `delete_course()` 内以 `conn=conn` 调用，并入同一事务，保证「先删融合行、再删 `t_course`」原子生效；④ `CourseService.delete_course()` 返回体补上 `removed_fusion` 分表计数（对齐文档级路径早就有的 `removed_fusion`）；⑤ 回归测试 `backend/test_fusion_course_delete.py`（18 项，含「对照课程一行不少」的越界检查）。
>
> **教训**：文档级与课程级是**两条独立的代码路径**，新增子表必须各挂一次——只挂文档级，删文档时正常、删课程时静默漏，且因为不报错而极难发现。

三、枚举取值（模块常量）

`sql_database.py` 顶部定义，供应用层校验（表内以 `CHECK` 约束保留同一取值集合）：

常量	取值
USER_ROLES	**admin**, teacher, student
DOC_FILE_TYPES	PDF, TXT, DOCX, MD
DOC_PARSE_STATUS	UPLOADED, PARSING, PARSED, FAILED
DOC_EXTRACT_STATUS	PENDING, EXTRACTING, COMPLETED, FAILED
RECORD_STATUS	MASTERED, LEARNING, RECOMMENDED
RECORD_SOURCE	MANUAL, SYSTEM
MEMBER_ROLES	teacher, student
MEMBER_STATUS	pending, approved, rejected, removed
MEMBER_JOIN_SOURCE	create, code, invite, apply, import
JOIN_MODES	auto（直接加入），approval（审核后加入），closed（关闭加入）
INVITE_STATUS	active, used, revoked
GENDERS	male, female, other, unknown
QUESTION_TYPES	SINGLE, MULTI, JUDGE, FILL, ESSAY

常量	取值
AUTO_GRADE_TYPES	SINGLE, MULTI, JUDGE（提交即判分）
MANUAL_GRADE_TYPES	FILL, ESSAY（提交只落库，教师批改后才给分）
ANSWER_GRADE_STATUS	PENDING（待批改）, GRADED（已判分）
QUESTION_SOURCE	MANUAL, AI, IMPORT
ANSWER_GRADE_SOURCE	AUTO, LLM, TEACHER
COURSE_GOVERNANCE_STATUS	normal（正常）, hidden（平台下架）, archived（归档）
USER_ACTIVE_STATUS	active, disabled
ADMIN_ACTIONS	管理员审计动作名元组，与 `api/admin.py` 调用点一一对应（详见其定义处注释）
COURSE_VISIBLE_SQL	`status = 1 AND COALESCE(governance_status, 'normal') = 'normal'` —— 课程对外可见性的**唯一判据**

> `JOIN_MODES` 取值以代码为准：`("auto", "approval", "closed")`，语义分别是「直接加入 / 审核后加入 / 关闭加入」。
>
> `USER_ROLES` 自管理员端起含 `admin`；旧库的 `t_user.role` CHECK 由 `_migrate_admin_role()` 重建补齐（见 §2.1）。
>
> **默认账号**：教师端引导账号 `admin`，管理员端账号 `sysadmin`（初始密码均为 `admin123`，仅演示/初始开发用，生产环境应删除或改为环境变量注入）。**新引导创建的账号会带 `must_change_password=1` 强制改密**（见 §2.1）——即「初始密码 `admin123`」只在首次登录前成立。

知识融合的枚举**没有集中常量**，直接内联在各自表的 `CHECK` 里（§2.16–2.20），汇总如下：

出处	取值
`t_kp_fusion_run.mode`	DRY_RUN（试算）, APPLY（正式应用）
`t_kp_fusion_run.status`	RUNNING, COMPLETED, FAILED, REVOKED
`t_kp_fusion_candidate.decision`	SAME（同义）, DIFFERENT（异义）, UNCERTAIN（不确定）
`t_kp_fusion_candidate.decision_source`	RULE（规则）, LLM（大模型）, MANUAL（人工）
`t_kp_fusion_candidate.status`	PENDING, ACCEPTED, REJECTED, DEFERRED, SUPERSEDED
`t_kp_fusion_conflict.conflict_type`	CYCLE（成环）, MULTI_TARGET（多目标）, ENDPOINT_MISSING（端点缺失）, CROSS_DOC（跨文档）
`t_kp_fusion_conflict.status`	OPEN, RESOLVED
`t_kp_fusion_map.status`	ACTIVE, REVOKED

> 阈值类参数（非枚举）集中在 `backend/app/services/fusion/fusion_config.py`（`AUTO_SAME_THRESHOLD`、`REVIEW_THRESHOLD`、`RECALL_THRESHOLD` 及各评分权重），并**快照进** `t_kp_fusion_run.config_json` 以保证批次可复现。

四、Neo4j 图模型

4.1 节点 KnowledgePoint

属性均全量存在（**658/658 节点**，2026-10-08 实查）：

属性	类型	说明
kp_id	string	业务主键，格式 `kp_{course_id}_{uuid}`
name	string	知识点名称
category	string	类别 ∈ {概念, 定理, 公式, 方法}
description	string	描述
course_id	integer	所属课程
document_id	integer	所属文档（文档级图隔离，路径推荐与问答按此过滤）
confidence	float	置信度（模型自评）
is_manual	boolean	是否教师手动创建
created_at / updated_at	string	时间戳

> 早期设计中的 `aliases` / `source_doc_ids` 两个属性**当前并未实际写入**，勿依赖。

4.2 关系类型

Cypher 类型	中文标签（relation_type 属性）	语义	当前数量
PRECEDES	前置知识	学习依赖（最高精度要求）	17
CONTAINS	包含	上下位/整体-部分	228
RELATED_TO	相关概念	严格限制	200
APPLIES_TO	应用	方法→应用对象	63
合计			**508**

关系属性：`relation_type`（中文标签）、`course_id`、`document_id`、`is_manual`、`created_at`、`updated_at`（以上 **508/508 全量**）；`confidence`（**507/508**）；`evidence`（证据，**507/508**，仅 LLM 抽取的关系带）。

> **数量是「当前库的快照」，会随文档重新抽取整体变动，不要当成设计常量引用。** 对比 2026-09-23 版（270 节点 / 302 关系）：节点 270 → 658，关系 302 → 508；且**分布不是等比增长**——`CONTAINS` 116 → 228、`RELATED_TO` 68 → 200（近 3 倍），而 `PRECEDES` 26 → **17**、`APPLIES_TO` 92 → **63** 反而**减少**。两个方向同时出现说明这批文档是**被重新抽取**而非追加导入（重新抽取会整体更换 `kp_id`，旧边被删除）。`evidence` 覆盖率同期从 137/302（45%）升到 507/508（99.8%），是抽取链路补全证据字段的结果。
>
> 唯一 1 条缺 `confidence` / `evidence` 的关系是历史残留，非系统性缺失。
>
> 早期设计中的 `is_inferred` 属性**当前并未实际写入**，勿依赖。

4.3 约束与索引

类型	名称	定义
唯一约束	`kp_id_unique`	`(kp:KnowledgePoint) REQUIRE kp.kp_id IS UNIQUE`
复合索引	`kp_course_name`	`(kp.course_id, kp.name)`
复合索引	`kp_course_category`	`(kp.course_id, kp.category)`
复合索引	`kp_course_doc`	`(kp.course_id, kp.document_id)` ——文档级作用域
复合索引	`kp_course_doc_name`	`(kp.course_id, kp.document_id, kp.name)` ——文档级 MERGE 键
关系索引	`precedes_course`	`[r:PRECEDES]` 的 `r.course_id`（路径推荐过滤用）

均为 `IF NOT EXISTS` 幂等创建，由 `init_schema()` 在后端启动时执行。

> 2026-10-08 实查（`SHOW CONSTRAINTS` / `SHOW INDEXES`）：以上 **6 项与文档完全一致**，无增无减。图库中另有 2 个 Neo4j 5 自带的 `LOOKUP` 索引（`index_xxx`，全库标签/属性查找用），非本项目创建。
>
> **知识融合模块对 Neo4j 零变更**：不新增标签、不新增关系类型、不新增属性、不删任何节点或关系——融合结果完全靠 SQLite 侧的 `t_kp_fusion_map` 在读图时折叠生效（见 §2.18）。这也是本次核对中「图模型结构一字未动、只有数据量翻倍」的原因。

五、孤岛模块（未挂路由）

`services/git_kg_manager.py` 中定义了 3 个 Git 分析用 Neo4j 标签与唯一约束，**当前无任何 API 挂载、前端无界面、计划书未提及**：

标签	唯一约束
GitCommit	`git_commit_hash`：`hash` 唯一
GitFile	`git_file_path`：`path` 唯一
GitAuthor	`git_author_email`：`email` 唯一

> 2026-10-08 实查确认**仍然如此**：运行库中只有 `KnowledgePoint` 一种标签，上述 3 个 Git 标签**一个都不存在**（约束也未创建）。即这段代码至今未被任何路径执行过，与本节的「孤岛」判断一致。

六、类型映射（MySQL 迁移参考）

SQLite 无 ENUM / UNSIGNED / ON UPDATE 语法，用等价手段保留语义：

MySQL	SQLite
BIGINT / INT / TINYINT UNSIGNED	INTEGER
VARCHAR / CHAR / TEXT / ENUM	TEXT（ENUM 取值用 CHECK 约束保留）
DATETIME	TEXT（存 `YYYY-MM-DD HH:MM:SS`）
AUTO_INCREMENT	INTEGER PRIMARY KEY AUTOINCREMENT
DEFAULT CURRENT_TIMESTAMP	DEFAULT (datetime('now','localtime'))
ON UPDATE CURRENT_TIMESTAMP	应用层 UPDATE 时显式写 updated_at
BLOB / LONGBLOB	BLOB（`t_kp_embedding.embedding_blob` 等存 float32 小端向量；MySQL 侧同样用 BLOB，不转 JSON）

七、迁移机制（无 Alembic）

项目未引入迁移框架，schema 演进靠 `SQLDatabase.init_tables()` 在**每次后端启动**时执行，全程幂等：

```
init_tables()
├─ 逐条执行 _SCHEMA_SQL                # 全部 CREATE TABLE / CREATE INDEX IF NOT EXISTS（22 张表）
├─ _migrate()
│  ├─ 1) document_id 补列与回填 + idx_kp_emb_scope
│  │                                   # t_learning_record / t_student_favorite / t_kp_embedding
│  ├─ 2) _migrate_question_bank()      # 题库：t_answer_record 批改列 + t_question 导入列
│  ├─ 3) _migrate_question_kp()        # L2：t_question.kp_id → t_question_kp 主知识点回填
│  ├─ 4) _migrate_embeddings_blob()    # L2 阶段 E：向量表补 embedding_blob + 限量回填
│  ├─ 5) _migrate_course_center()      # 课程中心改造（join_code 等 6 列）
│  ├─ 6) _migrate_admin()              # 管理员端：t_course 补治理三列
│  ├─ 7) _migrate_admin_user_columns()         # t_user 补 must_change_password
│  └─ 8) _migrate_document_extract_timing()    # t_document 补抽取起止时刻两列
├─ _migrate_question_types()           # t_question 重建（扩 q_type CHECK；需独立连接）
├─ _migrate_admin_role()               # t_user 重建（扩 role CHECK 含 admin；需独立连接）
└─ _apply_journal_mode()               # 阶段 G：最后切 WAL（库文件已非空才可设 WAL）
```

> **融合的 5 张新表不在此图内**：它们不需要任何迁移步骤，全由 `_SCHEMA_SQL` 的 `CREATE TABLE IF NOT EXISTS` 建出（见 §2.16–2.20）。

`_migrate()` 的各段都是「先探测、再变更」，重复执行结果一致：

1. **`document_id` 补列**：`PRAGMA table_info` 守卫，缺列才 `ALTER TABLE ADD COLUMN`；随后仅回填 `document_id IS NULL` 的行（历史数据「一课程 == 一文档」，可安全映射）。
2. **`_migrate_question_bank()`**：给 `t_answer_record` 补「批改」相关列（`_ANSWER_NEW_COLUMNS`；`grade_status` 默认 `GRADED`，保证历史记录口径不变），并给 `t_question` 补 Scope D 的导入批次/暂存状态列（`_QUESTION_NEW_COLUMNS`）——后者必须**先于** `_migrate_question_types()` 的重建，才能被一起拷过去。
3. **`_migrate_question_kp()`**（L2）：把 `t_question.kp_id` 回填为 `t_question_kp` 的 `is_primary=1` 行。**判据必须是** `NOT EXISTS (SELECT 1 FROM t_question_kp k WHERE k.question_id = q.question_id)`——只回填「关联表中无任何行」的题；若只靠 `INSERT OR IGNORE`，已有主知识点的题会被再插一行 `is_primary=1`，同时触碰 `uq_qkp_primary` 部分唯一索引，导致**后端每次启动即崩**。
4. **`_migrate_embeddings_blob()`**（L2 阶段 E）：给 `t_kp_embedding` / `t_question_embedding` 补 `embedding_blob BLOB`，并把 `embedding`(JSON) **限量**转成 float32 BLOB（每次启动 ≤ 5000 行）。限量是刻意的：5 万条 JSON 全量解析约 18 秒，会拖慢启动；重复启动幂等续跑，`backfill_embedding_blobs()` 可显式全量执行。
5. **`_migrate_course_center()` 四步**：
   - `PRAGMA table_info(t_course)` 守卫，逐个补 `_COURSE_NEW_COLUMNS` 中的 6 列；
   - `CREATE UNIQUE INDEX IF NOT EXISTS uq_course_join_code`；
   - 仅为 `join_code IS NULL` 的课程生成加课码（冲突重试至多 20 次）——**教师已发出的码不会被重新生成**；
   - `INSERT OR IGNORE INTO t_course_member ... SELECT ... FROM t_course`，把每门已有课程的 `teacher_id` 变成 `role='teacher'/status='approved'` 成员。靠 `UNIQUE(course_id, user_id)` 保证幂等，且不覆盖教师后续手动改动过的成员行。
6. **`_migrate_admin()`**（管理员端）：给 `t_course` 补 `governance_status` / `governance_note` / `archived_at` 三列（`PRAGMA table_info` 守卫），历史课程一律 `normal`。**同样不写进 `_SCHEMA_SQL`**——新库也走这段迁移，因此两库结构一致，且三列都不带 CHECK，取值由应用层 `COURSE_GOVERNANCE_STATUS` 白名单兜底。
7. **`_migrate_admin_user_columns()`**：给 `t_user` 补 `must_change_password`（`DEFAULT 0`）。历史行取 0 是刻意的——若取 1，一次升级会把所有在线用户挡在改密页上；该标记只对**新引导创建**的账号以及**被管理员重置过密码**的账号置 1。
8. **`_migrate_document_extract_timing()`**：给 `t_document` 补 `extract_started_at` / `extract_finished_at`（均可空 `TEXT`，**不伪造历史值**，旧行留 `NULL`），修复「用 `updated_at` 算抽取耗时」导致显示 48889 分的问题。详见 §2.6。

四条重要约束：

- 新增列**只写** `_migrate()`**、不写** `CREATE TABLE`。若两边都写，新建库与迁移库的表结构会分叉（列顺序、默认值不一致）。

- SQLite 的 `ALTER TABLE ADD COLUMN` **不允许带 UNIQUE / PRIMARY KEY**。`join_code` 若直接写成 `ADD COLUMN join_code TEXT UNIQUE`，后端会**启动即崩**。
- **新增「表」时相反：写进 `_SCHEMA_SQL` 的 `CREATE TABLE IF NOT EXISTS`**（连同 `CREATE INDEX IF NOT EXISTS`，天然幂等，不需要 ALTER）。`t_question_kp` 与 `uq_qkp_primary`（部分唯一索引）即按此办理。
- 新表若带**物理外键指向 `t_question`**，须注意两点：① `_migrate_question_types()` 会 DROP/重建 `t_question`（该连接内先关 `foreign_keys`，末尾用 `PRAGMA foreign_key_check` 兜底）；② 所有删除路径必须**先删新表的行、再删题目**——连接默认 `foreign_keys = ON`，顺序反了会直接抛 FK 约束错误。

> ⚠️ **约束一目前有两处已知例外**（2026-10-08 核对代码发现）：`t_user.must_change_password` 与 `t_document.extract_started_at` / `extract_finished_at` 这三列**同时写在 `_SCHEMA_SQL` 和 `_migrate()` 两处**。后果分两种：
>
> - `extract_started_at` / `extract_finished_at`：两处都是**无约束的可空 `TEXT`**，**不产生分叉**，属无害例外。
> - `must_change_password`：`_SCHEMA_SQL` 里**带** `CHECK (must_change_password IN (0, 1))`，`_migrate()` 的 `ALTER TABLE` 里**不带**（SQLite 的 `ADD COLUMN` 本就不允许带 CHECK）。**这会产生真实的结构分叉**——新建库该列有 CHECK，旧库迁移来的没有；取值合法性目前靠应用层兜底，不影响功能。
> - 另一个分叉点是 `t_user.role` 的 CHECK：`_SCHEMA_SQL` 版本含 `admin`，旧库靠 `_migrate_admin_role()` **重建表**补齐——这一处是**有意的**，机制见 §2.1。
>
> 核对新库 / 旧库表结构时遇到上述差异，属已知情况，不是「迁移没跑」。

历史课程不猜测学生成员关系：迁移只回填教师，学生需通过加课码 / 邀请 / 申请重新加入。

回滚方式：恢复 `backend/backup/app.db.pre_*.bak`；或按外键反序 DROP 新增表、DROP 索引、DROP 新增列。

八、连接与并发（pragma）+ 备份

8.1 连接 pragma（`_connect()`）

pragma	值	作用域	说明
`foreign_keys`	`ON`	连接级	级联与 FK 约束；**所有**删除路径依赖它，顺序错了会直接抛错
`busy_timeout`	`15000` ms	连接级	写者互斥时**等待**而非立刻报 `database is locked`。默认 `sqlite3.connect` 的 `timeout=5s` 对批量写不够
`journal_mode`	`WAL`	**库级、持久**	读不阻塞写、写不阻塞读（只有写-写串行）。只在 `init_tables()` 末端设置一次

为什么需要 WAL：部署形态是「一个 uvicorn 进程 + 若干 CLI 脚本/评测」共写同一个 `app.db`；rollback journal 模式下**长读事务会挡住写者提交**，而 L2 引入的批量写入（121 条向量 / 全量替换关联行 / ≤5000 行 BLOB 回填）又让写事务变长 —— 两头一夹就撞锁。

知识融合的 `apply` / `revoke` 同样把「写入映射 + `epoch+1` + 失效向量」压在一个事务里（见 §2.20），是又一个长写事务来源。

- 失败**不抛异常**：网络盘等不支持 WAL 时退回流式 journal，正确性不受影响；但状态会被 `sql_db.pragma_status()` 保留（`requested` / `actual` / `error`），不静默丢弃。
- 回退开关：`SQLITE_JOURNAL_MODE=delete`、`SQLITE_BUSY_TIMEOUT=0`。
- **边车文件**：WAL 会产生 `app.db-wal` / `app.db-shm`（运行期文件，已加入 `.gitignore`）。
- ⚠️ `backend/data/app.db` **被 git 跟踪**：WAL 下「主文件」可能不含最近提交，提交前应先让写事务结束/触发 checkpoint（最稳妥是出一份 `backup_to()` 快照再提交，或把该文件移出版本库）。这与 8.2 的裸拷问题是同一个坑。

8.2 备份必须走 `backup_to()`，不能裸拷文件

```
fact = sql_db.backup_to("backup/sqlite_<ts>.db")   # 官方 backup API，WAL 安全
```

为什么不能 `shutil.copy2(app.db)`：WAL 模式下**最近提交可能还留在** `app.db-wal` 里，裸拷主文件会得到「**静默少数据**」的备份（不报错，事后才发现）；拷贝恰逢 checkpoint 时还可能拿到内部不一致的快照。而回滚方式正是「恢复 `backend/backup/*.bak`」——**备份静默失效 = 回滚路径静默失效**。

`backup_to()` 的行为：`sqlite3.Connection.backup` 取**写事务边界上的一致快照**（不阻塞写者），并把备份文件 journal 复位为 `delete` 使其**单文件自包含**（无需连 `-wal` 一起拷）；返回逐表行数便于与线上核对。`scripts/backup_migration.py` 已改走它。

> 启用 WAL **之前**生成的 `app.db.pre_*.bak` / `sqlite_*.db` 仍然有效（rollback journal 下主文件本身完整）；**之后**必须用 `backup_to()`。